Functional Programming — Assignment 5

Student Name: Maxsutkhan Ramazanov | **Group:** IT-2506 | **Week:** 5
Assignment Title: Composing Functions Using Pipelining
GitHub Repository: https://github.com/poroshok13/funcprog5

1. Task 1: Currying and Partial Application

```
public static Func<A, Func<B, C>> Curry<A, B, C>(Func<A, B, C> f)
{
    return a => b => f(a, b);
}
```

```
var curriedDiscount = Curry<decimal, decimal, decimal>(ApplyDiscount);
var curriedFee = Curry<decimal, decimal, decimal>(AddFee);

Func<decimal, decimal> discount10 = curriedDiscount(0.10m);
Func<decimal, decimal> fee5 = curriedFee(5m);
```

Currying turns a binary function `(A, B) → C` into a unary chain `A → B → C`. By providing the configuration parameter first (`rate = 0.10m`, `fee = 5m`), we obtain unary functions `Func<decimal, decimal>` via partial application.

2. Task 2: Function Composition (Compose)

```
public static Func<A, C> Compose<A, B, C>(Func<A, B> f, Func<B, C> g)
{
    return x => g(f(x));
}
```

```
Func<decimal, decimal> checkout = Compose(Compose(discount10, fee5), round);
decimal result = checkout(1234.56m); // 1116.10
```

`Compose` performs forward composition: it executes `f` first and passes its result into `g`. Composing `discount10`, `fee5`, and `RoundToCents` creates a single reusable `checkout` function.

3. Task 3: Pipelining Extension Method (Pipe)

```
public static class PipeExtensions
{
    public static R Pipe<T, R>(this T value, Func<T, R> f)
    {
        return f(value);
    }
}
```

```
decimal result = 1234.56m
    .Pipe(discount10)
    .Pipe(fee5)
    .Pipe(round); // 1116.10
```

The extension method `Pipe` allows writing calculations from left to right, eliminating nested parentheses and matching the natural data-flow order.

4. Task 4: Dynamic Pipeline with Aggregate

```
public static Func<decimal, decimal> Pipeline(params Func<decimal, decimal>[] ste
{
    return steps.Aggregate(
        (Func<decimal, decimal>)(x => x),
        (acc, step) => x => step(acc(x))
    );
}
```

Pipeline Configuration	Sequence of Functions	Raw Decimal Output	Explanation
1. Standard	discount10 → fee5 → round	`1116.10`	10% discount on order only; fee added in full afterwards; cents rounded.
2. Swapped	fee5 → discount10 → round	`1115.60`	Fee added first; 10% discount reduces the fee by 0.50 (5 × 0.10).
3. Round First	round → discount10 → fee5	`1116.1040`	Rounding executed first; discount produces unrounded fractional cents.
4. Empty Pipeline	Pipeline()	`1234.56`	Returns identity function `x => x`; input value is unchanged.

5. Console Output

```
Assignment 5 - Composing Functions Using Pipelining
Maxsutkhan Ramazanov | IT-2506

--- Task 1: Currying & Partial Application ---
Initial Amount: 1234,56
After 10% discount: 1111,1040
After 5 flat fee: 1116,1040
After rounding: 1116,10

--- Task 2: Function Composition ---
Order amount: 1234,56
Checkout result: 1116,10

--- Task 3: Pipelining with Extension Method ---
Order amount: 1234,56
Piped result: 1116,10
```

```
--- Task 4: Dynamic Pipeline with Aggregate ---
Input amount: 1234,56
1. Standard (discount -> fee -> round): 1116,10
2. Swapped (fee -> discount -> round):   1115,60
3. Round First (round -> discount -> fee): 1116,1040
4. Empty Pipeline ():                    1234,56
```

6. Answers to Questions

Question 1: Pure Functions in Composition

- **Why required:** Composition relies on mathematical associativity and referential transparency. If a step has side effects (modifies state, writes to database, depends on time), reordering steps or evaluating the pipeline multiple times causes unintended side effects and non-deterministic behavior.
- **Payment system example:** A step `ChargeCreditCard(amount)` inside the pipeline would charge real money each time the pipeline is evaluated (e.g. during price preview or retry).

Question 2: Currying vs Partial Application & Parameter Ordering

- **Difference:** Currying transforms a function of *n* arguments into a chain of *n* unary functions (f(a, b) → a → b → c). Partial application fixes one or more arguments upfront, returning a function with fewer parameters.
- **Parameter ordering:** Configuration values (`rate` , `fee`) are placed first, and the data operand (`amount`) last. This allows partially applying configuration once to get a unary function `Func<decimal, decimal>` ready for pipelining.

Question 3: Order Differences and Rounding Position

- **Why results differ:** Multiplication and addition do not commute: (x − 0.1x) + 5 ≠ (x + 5) × 0.9. Applying the fee first discounts the fee as well, saving .50.
- **Rounding first:** Rounding the input upfront has no effect on whole cents, but subsequent percentage discounting produces fractional cents (`1116.1040`). Rounding must always be the final step to eliminate intermediate fractional cents.

Question 4: Compose, Pipe, Pipeline and Identity Seed

- **Advantages over nested calls:** `Compose` creates reusable composite functions; `Pipe` provides clean left-to-right reading order on values; `Pipeline` allows building steps dynamically from runtime collections.
- **Return value and seed:** `Pipeline()` without arguments returns the identity function `x => x` . The identity function is the neutral element of composition (f ∁ id = id ∁ f = f), ensuring `Aggregate` handles an empty sequence safely without throwing exceptions.